Report

- Chat with AI
- Problems
  - Static Latency Model
  - Dependency Detection
  - Priority Calculation
  - Tie-Breaking Policies
  - Complexity Analysis
  - End-to-End Integration
  - Results and Analysis
    - Example 1: Expression Calculation
    - Example 2: Dependency Type Demonstration
    - Example 3: Load-Use Hazard Hidden
    - Example 4: Early Start Critical Operation

Chat with AI

1. https://chat.deepseek.com/share/lv0f846jlm0x66t8jx
2. https://chatgpt.com/share/69cb4893-4944-839a-a3ed-9421676710bc

Problems

Note: for implementing functions, I had my thoughts written in GoodNotes for the planning of what to do. I used chatf I'm unsure of something, I will ask AI, and the conversations can be found above. After being satisfied with it, I try to code it based on the workings I've written. When forgetting the syntax I turn to https://www.geeksforgeeks.org/cpp/c-plus-plus/, https://www.geeksforgeeks.org/cpp/max-heap-in-cpp/ , https://www.geeksforgeeks.org/cpp/priority-queue-in-cpp-stl/ ,

I apologize in advance for any messy handwriting, as this is the learning process that works for me.

Static Latency Model

Problem (Static latency model [2+2+5 point]):

(a) We know both load and store are memory instructions, why in static latency model we can assume load have 5 clcyle latency while store only take 1 cycle latency?
Deliverable: A one to two sentence responce

(b) Why a static latency model makes sense even though real instructions do not always take a fixed number of cycles?
Deliverable: A one to two sentence responce

(c) There are some code missing in the `getInstructionLatency()` function, please fix it
Deliverable: Fix `getInstructionLatency()` (src/instruction.cpp)

To test your function on our provided test, first `cd build` and `make -j4` then run `ctest -R LatencyTest` You Implementation should be able to pass all tests.

(a) In a 5-stage pipeline with IF ID EX MEM and WB, a load instruction would have to go through all 5 stages, since the result of a load becomes available only after the WB stage (assuming no forwarding), and each stage takes one cycle, any dependent instruction must wait 5 cycles. Stores are estimated 1 cycle because they don't create a value that later instructions will need through RAW dependency.

(b) A static latency model just gives a simplified, predictable baseline for the scheduler to reorder instructions, assuming cache hits, while ignoring cache misses that are difficult to predict at compile time. This causes consistent performance improvements and keeps the scheduling algorithm practical and portable across different microarchitectures.

(c) I only noticed and added the below missing code:

mul under multiplication

```
{    "mul",  3}, // added
```

rem under Division

```
{    "rem", 10}, // added
```

lw under load

```
{    "lw",  5}, // added
```

sw under store

```
{    "sw",  1}, // added
```

That's all.

Dependency Detection

Problem (Dependency Detection [2+2+10+10+[5] point]):

(a) Why there is no RAR Dependency?
Deliverable: A one to two sentence responce

(b) Why we say Dependency Graph should be a DAG, what if not?
Deliverable: A one to two sentence responce

(c) Implement WAW and WAR detection
Deliverable: `detectWAR()`, `detectWAW()` (src/DAGBuilder.cpp)

(d) **Bonus**: If you read `DAGBuilder:build`, you will find out detectWAW/WAR/RAW can be done together, can you do it in one pass?
Deliverable: one pass function to build DAG

To test your function on our provided test, first `cd build` and `make -j4` then run

```
ctest -R "DAGBuilder|Integration"
```

You Implementation should be able to pass all tests.

For the Bonus Part, you can omit previous `detectXXX` functions we provided and rewrite the `DAGBuilder:build` by yourself, make sure it still pass all test case above.

a) Because reading doesn't need to be one in any specific order, it doesn't modify the state of the register.

b) It should be a directed acyclic graph, without directions, we won't be able to know which instructions (modeled as the nodes) should come first/after. Additionally, having cycles in a dependency graph means it's a circular dependency, and we can't find a valid topological sort (instruction order) that doesn't violate any dependencies.

c) I've decided to implement the Bonus one pass in build()

along with these 2 headers

```
#include <unordered_map>
#include <unordered_set>
```

```
// // the old multi-pass implementation
// detectRAW(instructions, nodes);
// detectWAR(instructions, nodes);
// detectWAW(instructions, nodes);

// ONEPASS DETECTION, the bonus part
std::unordered_map<std::string, int> lastWriter;
std::unordered_map<std::string, std::unordered_set<int>> readersSinceLastWrite;

for (size_t i = 0; i < instructions.size(); i++) {
    const Instruction& inst = instructions[i];
    const int currentIdx = static_cast<int>(i);

    // RAW: current source depends on the last writer of the register we read from
    for (const auto& srcReg : inst.src_regs) {
        auto it = lastWriter.find(srcReg);
        if (it != lastWriter.end()) {
            addEdge(nodes, it->second, currentIdx, DependencyType::RAW, srcReg);
        }
    }

    if (inst.hasDestReg()) {
        const std::string& dest = inst.dest_reg;

        // WAW: current write depends on previous write in order
        auto writerIt = lastWriter.find(dest);
        if (writerIt != lastWriter.end()) {
            addEdge(nodes, writerIt->second, currentIdx, DependencyType::WAW, dest);
        }

        // WAR: current write depends on all readers since the last write
        auto readersIt = readersSinceLastWrite.find(dest);
        if (readersIt != readersSinceLastWrite.end()) {
            for (int readerIdx : readersIt->second) {
                addEdge(nodes, readerIdx, currentIdx, DependencyType::WAR, dest);
            }
            readersIt->second.clear();
        }

        lastWriter[dest] = currentIdx;
    }

    // record what reg this instruction reads
    for (const auto& srcReg : inst.src_regs) {
        readersSinceLastWrite[srcReg].insert(currentIdx);
    }
}
```

I first implemented detectWAR() and detectWAW() as separate functions to make the logic easier to reason about and test. After that, I rewrote build() using the bonus one-pass approach, where RAW, WAR, and WAW are all detected. The separate versions were useful for developing my understanding, but the **final submitted implementation uses the one-pass** build() version.

```
// // the old multi-pass implementation
// detectRAW(instructions, nodes);
// detectWAR(instructions, nodes);
// detectWAW(instructions, nodes);
```

I have the above commented out in my submission.
For the Individual detectWar() my thought process is:

For WAR: current write depends on all readers
1. Need map to track readers since last write to a specific reg.
2. For each instruction need to check if it writes
   aka. check if there's destination.
   ↓ if have dest.
3. Find the register if it exists in the reader map.
   if it exists then there's a set of indexes (ints) of instruction that needs to read this register before it gets written.
4. Add edge between each index in the set to our current instruction with dependency type WAR.

```
void DAGBuilder::detectWAR(const std::vector<Instruction>& instructions,
                           std::vector<DAGNode>& nodes) {
    Implement with Codex
    // TODO: Implement this function
    // Hint:
    // You need Track all readers of each register (after the last write)
    // Iterate each instruction, If current instruction writes a register, check previous readers
    // Add WAR dependency to all previous readers
    // WAR latency is 0 because read happens at instruction start
    // throw std::runtime_error("WAR dependency detection not implemented");

    // track readers observed since the last write for each register.
    std::map<std::string, std::set<int>> readersSinceLastWrite;

    for (size_t i = 0; i < instructions.size(); i++) {
        const Instruction& inst = instructions[i];
        const int currentIdx = static_cast<int>(i);

        if (inst.hasDestReg()) {
            const std::string& dest = inst.dest_reg;

            auto it = readersSinceLastWrite.find(dest);
            if (it != readersSinceLastWrite.end()) {
                for (int readerIdx : it->second) {
                    addEdge(nodes, readerIdx, currentIdx, DependencyType::WAR, dest);
                }
                it->second.clear();
            }
        }

        for (const auto& srcReg : inst.src_regs) {
            readersSinceLastWrite[srcReg].insert(currentIdx);
        }
    }
}
```

For the detectWAW() my thought process is:

For WAW: curr. write depends on previous write in order
1. need map to track register name with instruction that wrote it last
2. For each instruction
   check if writes to register:
   if no write skip to next inst.
   if write then WAW is possible!
   ↓
3. So need to check if our destination register exists in our map.
   if it exists, that means a previous instruction has written to it
4. Add edge from previous instruction (found from finding the match in 3) then add dependency from it to our current instruction with type WAW.

index → second - - - - → static_cast<int>(i)

```
void DAGBuilder::detectWAW(const std::vector<Instruction>& instructions,
                           std::vector<DAGNode>& nodes) {
    Implement with Codex
    // TODO: Implement this function
    // Hint:
    // Track the last instruction that wrote to each register
    // WAW latency is 1, only need to guarantee order
    // throw std::runtime_error("WAW dependency detection not implemented");

    std::map<std::string, int> lastWriter;

    for (size_t i = 0; i < instructions.size(); i++) {
        const Instruction& inst = instructions[i];
        if (!inst.hasDestReg()) {
            continue;
        }

        auto it = lastWriter.find(inst.dest_reg);
        if (it != lastWriter.end()) {
            addEdge(nodes, it->second, static_cast<int>(i), DependencyType::WAW, inst.dest_reg);
        }

        lastWriter[inst.dest_reg] = static_cast<int>(i);
    }
}
```

```
std::unordered_map<std::string, int> lastWriter;
std::unordered_map<std::string, std::unordered_set<int>> readersSinceLastWrite;

for (size_t i = 0; i < instructions.size(); i++) {
    const Instruction& inst = instructions[i];
    const int currentIdx = static_cast<int>(i);

    for (const auto& srcReg : inst.src_regs) {
        auto it = lastWriter.find(srcReg);
        if (it != lastWriter.end()) {
            addEdge(nodes, it->second, currentIdx, DependencyType::RAW, srcReg);
        }
    }

    for (const auto& srcReg : inst.src_regs) {
        readersSinceLastWrite[srcReg].insert(currentIdx);
    }

    if (inst.hasDestReg()) {
        const std::string& dest = inst.dest_reg;
        auto writerIt = lastWriter.find(dest);
        if (writerIt != lastWriter.end()) {
            addEdge(nodes, writerIt->second, currentIdx, DependencyType::WAW, dest);
        }
        auto readersIt = readersSinceLastWrite.find(dest);
        if (readersIt != readersSinceLastWrite.end()) {
            for (int readerIdx : readersIt->second) {
                addEdge(nodes, readerIdx, currentIdx, DependencyType::WAR, dest);
            }
            readersIt->second.clear();
        }

        lastWriter[dest] = currentIdx;
    }
```

Note for One pass,
RAW must be checked first
↓
WAW and WAR use maps before lastWriter is updated and before adding the current inst. to readersSinceLastWrite.
↓
Update lastWriter
↓
Add current instruction to reader sets.

In an earlier version of the one-pass builder, my update order was wrong. The main bugs were:

1. I **recorded reads too early**, so the current instruction could be included in WAR checking of its own destination, e.g. add x1, x1, x2, creating an incorrect self-dependency.
2. I **did not clear the destination's reader** set after WAR handling, so later writes could still see not prevalent readers and produce invalid WAR edges.

Fix: I fixed this by reordering the loop: RAW first, then WAW, then WAR, then clear readers for the destination, update lastWriter[dest], and only then record the current instruction's reads.

Priority Calculation

Problem (Priority Calculation [4+10+10 point]):

(a) Why WAR and WAW dependencies only enforce ordering (i.e. no timing delay)
Deliverable: A one to two sentence responce

(b) Implement the critical path length calculation for scheduling priority.
Deliverable: Implement `computeCriticalPath()` (src/Scheduler.cpp)

(c) Implement List Scheduling process with an inflight queue. Your implementation must correctly distinguish between RAW and WAR/WAW edges: WAR/WAW successors are released when the predecessor is *issued*; RAW successors are released only when the predecessor *finishes* (i.e., at $issue_cycle + latency$).
Deliverable: Implement `schedule()` (src/Scheduler.cpp)

To test:

```
cd build && make -j4
ctest -R
"SchedulerTest.ComputePriorities|SchedulerTest.Schedule_"
```

(a) It is because for WAR/WAW its not true data dependency, the instructions with WAR/WAW dependency don't need to wait for any result or computed value to become ready. In WAR, the earlier instruction just needs to read the register before it gets overwriten, and in WAW the earlier instruction needs to occur before the later write, for both cases, after the first instruction is issued, the dependency is already satisfied.

(b) Before trying to implement it I tried to write out algorithm 2 from the pdf again so I can understand it better.

Algorithm 2.

1. All instruction w/ no incoming dependency is in ready
2. At each cycle if have ready instruction, pick highest priority one, issue it and move it to inflight. For each WAW WAR successor, decrement its unscheduled predecessor count if count reach 0, become ready.
3. Increment cycle
4. For any instruction that finishes the cycle (cycle = issue_cycle + latency) remove it from inflight. For each RAW successor, decrement its unscheduled predecessor count, if 0 then becomes ready.
5. Repeat until all instructions scheduled.

Critical Path → max cycles from that node to the end Longest chain of dependent instructions!
→ To tell how urgent to schedule the node.

For example:

```
1. lw   x1, 0(x2)     // lat 5
2. mul  x3, x1, x1    // lat 3  depends on 1 by RAW
3. add  x4, x3, x3    // lat 1  depends on 2 by RAW
4. sw   x4, 0(x5)     // lat 1  depends on 3 by RAW
```

visited [1 ... 4] initially all false
priority [1 ... 4] initially undefined

Call computeCriticalPath (1)
visited [1] false so proceed
mark visited [1] as true
best = latency (1) = 5
Node 1 has 1 successor (Node 2) via RAW edge RAW w = latency

I tried to go over 4 example instructions, and think about what I want computeCriticalPath() to do when I called it. DeepSeek gave me a tip to do memoization:

What is memoization?

Memoization is an optimization technique used in recursive functions: you store the results of expensive function calls and return the cached result when the same inputs occur again.

le 1 has 1 successor (Node 2) via RAW edge RAW w = latency
# of cycles from start of current instruction until end w is
NOT RAW w = 1 Instruction start next cycle
priority of sucessor subpath
best is originally latency, need to track max (best, w + sub) Update best so it is max over all paths

I needed to calculate the **w** which is the number of cycles after the current instruction starts until the next can start. then also add in the subpath (**sub**)'s value which is the priority of the successor.

```
w = (RAW) ? latency(1) : 1 = 5
Then recursively call compute Critical Path (2)

compute Critical Path (2)
  visited [2] false, proceed
  mark visited [2] true
  best = latency (2) = 3
  Node 2 has 1 succesor (Node 3) via RAW edge
    w = latency (2) = 3
    recursively call compute Critical Path (3)

compute Critical Path (3)
  visited [3] false, proceed
  mark visited [3] true
  best = latency (3) = 1
  Node 3 has one successor (Node 4) via RAW edge
    w = latency (3) = 1
    recursively call compute Critical Path (4)
```

```
recursively call compute Critical Path (4)

compute Critical Path (4)
  visited [4] false, proceed
  Mark visited [4] true
  best = latency (4) = 1
  Node 4 no successors
  Store priority [4] = 1
  return 1 to caller (Node 3)

back in compute Critical Path (3)
  sub = 1 from above
  w + sub = 1 + 1 = 2
  best = max (1, 2) = 2
  store priority [3] = 2
  return 2 to node 2

back in compute Critical Path (2)
  sub = 2
  w + sub = 3 + 2 = 5
  best = max (3,5) = 5
  store priority [2] = 5
  return 5 to node 1

back in compute Critical Path (1)
  sub = 5
  w + sub = 5 + 5 = 10
  best = max (5, 10) = 10
  store priority [1] = 10
  return 10
```

- Node 1 (lw) has priority 10. That means from the moment lw starts, the block will take at least 10 cycles to finish. Indeed, the critical path is: lw(5) + mul(3) + add(1) + sw(1) = 10 cycles.
- Node 2 (mul) has priority 5: from mul start, it takes 5 cycles (mul 3 + add 1 + sw 1).
- Node 3 (add) priority 2 (add 1 + sw 1).
- Node 4 (sw) priority 1 (its own latency).

The implementation follows this example, where I traced what I wanted computeCriticalPath() to do.

```cpp
int Scheduler::computeCriticalPath(int nodeIdx,
                                   const std::vector<Instruction>& instructions,
                                   std::vector<DAGNode>& nodes,
                                   std::vector<bool>& visited) {
    if (visited[static_cast<size_t>(nodeIdx)]) {
        return nodes[static_cast<size_t>(nodeIdx)].priority;
    }
    visited[static_cast<size_t>(nodeIdx)] = true;

    const Instruction& inst = instructions[static_cast<size_t>(nodeIdx)];
    int best = inst.latency;

    for (const auto& edge : nodes[static_cast<size_t>(nodeIdx)].successors) {
        const int w =
            (edge.type == DependencyType::RAW) ? inst.latency : 1;
        const int sub = computeCriticalPath(edge.target_node, instructions, nodes, visited);
        best = std::max(best, w + sub);
    }

    nodes[static_cast<size_t>(nodeIdx)].priority = best;
    return best;
}
```

(c)

Schedule ()

result stores the schedule order (result.order) & total cycles (result.total_cycles)
if no instructions, return empty result

// init node state
reset each node's scheduling status
scheduled tells if the instruction has been issued
scheduled_cycle stores the order index
unscheduled_predecessors counts how many predecessors are not yet satisfied, as dependency ↓ count ↓

// Compute priority (critical path)
keep results in Vector to not let recursion compute many time (visited)
call computePriority to go through all nodes, for each unvisited call computeCriticalPath

// For scheduling
Finish event is a pair (finish_cycle, node_index)
Priority queue is a min-heap because std::greater makes smallest finish_cycle on top.
Need vector issue_time, issue_time[i] records the cycle when inst. i was issued

// Scheduling Loop
1. Retire finished instructions
Loop until all instructions scheduled
While there's an inst. in finishing queue that finishes at or before the current cycle t
pop it // current instruction is u
for each RAW successor of u (an inst. that reads result of u → decrement unscheduled_predecessors
2. Get ready instructions gets a vector of indices where !scheduled && unscheduled_predecessors == 0
3. If there are ready instructions
based on priority first then policy (original order)
record issue time, issue_time[k] = t
mark scheduled set its schedule order index, put in result
push k to finishing queue with finish time = current cycle + latency
t++
4. If no ready instructions
// inst. could be in inflight so just jump to earliest finish time (at top of finishing queue)
? If finishing is also empty (no flight, nothing ready) just break ?

// Count total cycles
it is max over all instructions of (issue time + latency) ⇒ time when last inst. finishes

The above is how schedule() works, and originally my algorithm didn't work because I was not supposed to process the WAW/WAR successors of u in the retirement step. Non-RAW successors are handled at issue time, not at finish time. The implemented code can be found along lines 9 - 79. It follows logic for list scheduling with an inflight queue: initialising node state, computing priorities, using a **minheap** for finish events, retiring finished instructions, issuing ready instructions, and calculating total cycles.

Tie-Breaking Policies

Problem (Tie-Breaking Policies [2+2+10 point]):

(a) Why might MOST_CHILD be a good tie-breaking strategy? Give a brief example scenario.
Deliverable: A one to two sentence response

(b) In what scenario would LPT outperform SMALLER_INDEX?
Deliverable: A one to two sentence response

(c) Implement all three tie-breaking policies in `selectBestNode()`, if two candidates still tie on MOST_CHILD or LPT, use SMALLER_INDEX to break the tie.
Deliverable: Implement tie-breaking in `selectBestNode()` (src/Scheduler.cpp)

To test: `cd build && make -j4 && ctest -R TieBreaking`

(a) If we schedule a node with many successors faster, it will unblock more dependent instructions sooner. For example, if a node computes a value that will be used in many ALU operations, scheduling it earlier allows it to finish earlier and allows the successors to become ready earlier.

(b) SMALLER_INDEX doesn't account for instruction latency, while LPT breaks ties according to latency, which tends to start long-latency instructions earlier. LPT outperforms SMALLER_INDEX when 2 ready instructions have same priority, but the shorter-latency one appears first in program order, by starting the long-latency instruction first, other independent instructions can execute while the long-latency is still in flight.

```
selectBestNode () : need to choose instruction w/ highest priority (critical path length)
                    if multiple inst. got same priority → do tie-breaking policy

SMALLER_INDEX : preserve program order
MOST_CHILD : get rid of node w/ most dependency first
LPT (Longest Processing Time) : prioritize highest latency first
if tie go with smaller idx.

selectBestNode ()
    just store first node as best, store index and the priority
    // Loop over all ready nodes
    note current node, get priority
  if compare with current best, if current > best → update best
    // tiebreak if priorities equal
  else if current == best Priority
      shouldReplace = false   its true if current should replace best
  switch (policy)
      case SMALLER_INDEX
          choose node w/ smaller original index
      case MOST_CHILD
          compare # of outgoing edges (successors) of the 2 nodes
          if current node has more successors, it replaces best
          if the have the same # → use SMALLER_INDEX
      case LPT
          compare instruction latencies (from instructions array)
          if current node has higher latency, replaces best
          if latencies equal → use SMALLER_INDEX
  if shouldReplace == true, update bestNode   best priority is same so no need update

  return bestNode
```

(c)

selectBestNode() first checks based on priority and uses the tie-breaking if they are equal. Its implementation can be found in 148 - 208.

Complexity Analysis

Problem (Complexity Analysis [2+2+[5] point]):

(a) What is the time complexity of the naive list scheduling implementation (the one that scans all nodes to find ready nodes each iteration)?
Deliverable: Express in Big-O notation with brief explanation

(b) What is the time complexity of priority calculation using the recursive memoized approach?
Deliverable: Express in Big-O notation with brief explanation

(c) **Bonus**: The current implementation rebuilds the ready list by scanning all nodes each iteration. How can we use a priority queue (max-heap for ready nodes, min-heap for inflight retirement) to optimize this? What would be the new time complexity? Can you give the code?
Deliverable: Brief description and complexity analysis; implement `scheduleFast()` (src/Scheduler.cpp). Your optimized version must also use an inflight queue with the same RAW/WAR/WAW release semantics as `schedule()`.

To test: `cd build && make -j4 && ctest -R SchedulerPerformance`

(a) The naive list scheduler takes **$O(n^2 + e)$** time because it might schedule up to n instructions in worst case, and in each iteration, it scans all n nodes to rebuild or find the ready set. For edge-processing, each WAR/WAW edge is checked when its predecessor is issued and each RAW edge is checked when its predecessor finishes, so in conclusion, each edge e is processed.

In my own schedule() implementation it runs in $O(n^2 + e + n\log n)$, the nlogn is from heap operations, and it also simplifies to $O(n^2 + e)$.

(b) The recursive memoized priority calculation runs in **$O(n + e)$** time because each node's critical path is computed only once due to memoization, and each edge is visited once while recursively exploring successors.

(c) scheduleFast() keeps the same RAW/WAR/WAW release methods like schedule(), but replaces repeated ready-list scans with a max-heap of ready nodes. Each node can enter the ready heap once and be popped once, so heap operations over nodes cost O(n log n). Dependency processing contributes O(e log n) in the worst case because when an edge release causes a successor's predecessor count to reach zero, that successor is inserted into the heap in O(log n) time. Therefore, the overall complexity improves to about **$O((n + e)\log n)$**. The implementation is in lines 82–199.

End-to-End Integration

Problem (Run Scheduler [10+10 point]):

(a) Schdule Code blocks in `main.cpp`, the following are some functions you should use:

- Parser::parse()
- DAGBuilder::build()
- Scheduler::schedule()
- Scheduler::printSchedule()
- Scheduler::printComparison()

Deliverable: Implement `runScheduler()` and analysis schedule results (src/main.cpp)

There are no test for `runSchedule`, instead you should analysis the result of example codes, does the scheduling improve performance? Why or Why not? Write your finding into report.

(a) The implementation can be found in 96 - 142.

runScheduler() connects all components of the project into one complete pipeline. It first parses the input assembly block into Instruction objects, then builds the dependency DAG, runs the scheduler to produce a reordered instruction sequence, and prints both the final schedule and the comparison against the original order. To support result discussion, it also constructs the original instruction order [0,1,2,...,n-1], simulates its execution, and compares its total cycle count with the scheduled result. Finally, it prints a short qualitative analysis

explaining whether the improvement came from overlapping independent work with long-latency operations, or whether dependency constraints already limited reordering opportunities.

I've done the following commands in my VS Code WSL terminal

```
cd build
make -j4
./scheduler > scheduler_out.txt
less scheduler_out.txt
```

Results and Analysis

Example 1: Expression Calculation

No improvement was observed because the original order was already close to optimal. Although lw x14 was moved earlier in the scheduled order, the total execution time still remained 16 cycles since the main computation was constrained by the dependent multiply-add chain.

Example 2: Dependency Type Demonstration

No improvement was possible because the block is dominated by strict dependencies between consecutive instructions. The chain of RAW, WAR, and WAW constraints leaves almost no freedom for useful reordering, so both orders take 7 cycles.

Example 3: Load-Use Hazard Hidden

Scheduling improved performance by 3 cycles, from 17 to 14 cycles, because the scheduler grouped the loads earlier and delayed their dependent multiplies until the load results were ready. This hides load-use latency by filling otherwise idle slots with independent memory operations.

Example 4: Early Start Critical Operation

Scheduling improved performance by 5 cycles, from 21 to 16 cycles, because the long-latency div was started immediately. This allowed the shorter add and multiply chain to execute in parallel with the division, reducing the overall critical-path completion time.

Overall, the results show that scheduling is most effective when independent instructions can be moved to overlap long-latency operations, but gives little or no benefit when the instruction block is already near optimal or heavily constrained by dependencies.